

Jobsheet 9 — CRUD Penuh

Dokumentasi ini melanjutkan

[dokumentasi jobsheet-08](#)

(Koneksi PostgreSQL). Jobsheet-09 **melengkapi** apa yang sudah dibangun sejak jobsheet-08 — kalau jobsheet-08 baru mencakup **Create** (tambah data) dan **Read** (tampilkan data), jobsheet ini menambahkan dua huruf terakhir dari **CRUD**: **Update** (ubah) dan **Delete** (hapus).

Tentang docs/wireframe.md

File ini **identik persis** dengan

[`docs/wireframe.md` di jobsheet-08](#) —

tidak ada rancangan UI/UX baru di jobsheet ini.

Apa yang Baru di Jobsheet 9?

Sesuai jobsheet ini:

1. **`edit.php` + `proses\_edit.php`** (buku & anggota) — melengkapi **Update**: form yang **sudah terisi** data lama, lalu menyimpan perubahannya.
2. **`hapus.php`** (buku & anggota) — melengkapi **Delete**, sengaja

hanya menerima `POST` (bukan `GET`) supaya tidak terpicu tidak sengaja lewat tautan biasa atau crawler mesin pencari.

3. Tombol Hapus di `list.php` sekarang berupa **`<form>` sungguhan** (bukan `<button>` polos seperti jobsheet-05/06) — `app.js` diubah menangani konfirmasi di event `submit`, bukan `click`.

4. **Pagination** (`LIMIT/OFFSET`, 5 baris per halaman) dan **pencarian sisi server** (`WHERE ... ILIKE ...`) — menggantikan pencarian client-side murni dari jobsheet-05/06 untuk kebutuhan mencari **lintas semua halaman**, bukan cuma baris yang sedang tampil.

Daftar Isi

5. [Konsep Dasar CRUD](#)
6. [Mengubah Data: `edit.php` & `proses\\_edit.php`](#)
7. [Menghapus Data: `hapus.php`](#)
8. [JS: Konfirmasi Hapus via Event `submit`](#)
9. [Pagination & Pencarian Sisi Server](#)
10. [CSS Pendukung Fitur Baru](#)
11. [Rangkuman & Latihan Lanjutan](#)

Struktur Folder

jobsheet-09/

```

├─ index.php
├─ includes/                # Tidak berubah dari jobsheet-08
├─ buku/
│   └─ list.php             # + pagination, pencarian server,
tombol Hapus jadi form
│   └─ tambah.php, proses_tambah.php # Tidak berubah dari jobsheet-08
│   └─ edit.php             # BARU – form edit terisi data
lama
│   └─ proses_edit.php      # BARU – UPDATE ke database
│   └─ hapus.php            # BARU – DELETE, hanya menerima
POST
├─ anggota/                # Struktur sama persis dengan
buku/
│   └─ list.php, tambah.php, proses_tambah.php
│   └─ edit.php, proses_edit.php, hapus.php
├─ assets/
│   └─ css/style.css        # + gaya btn-edit, pagination,
search form
│   └─ js/app.js            # initHapusConfirm: click →
submit
├─ docs/wireframe.md        # Identik dengan jobsheet-08

```

Catatan penting dari [README.md](#) jobsheet ini:

- Kolom pencarian sekarang melayani **dua peran sekaligus**: filter instan sisi klien (JavaScript, dari jobsheet-05) untuk baris yang **sedang tampil**, dan pencarian penuh **lintas-halaman** lewat tombol "Cari" (sisi server) — dibahas di [bab 5](#).

- Nilai pencarian (q) **belum di-escape** saat ditampilkan kembali ke atribut `value` — ini **sengaja belum diperbaiki**, akan diaudit dan

diperbaiki menyeluruh (bersama celah keamanan lain) di Jobsheet 11.

1. Konsep Dasar CRUD

1.1 Apa itu CRUD?

CRUD adalah singkatan dari 4 operasi dasar yang hampir **selalu** dibutuhkan aplikasi apa pun yang mengelola data:

Huruf	Operasi	Perintah SQL	Sudah Dibangun Sejak
Create	Menambah data baru	INSERT	Jobsheet 8 (dokumentasi jobsheet-08 §5)
Read	Membaca/menampilkan data	SELECT	Jobsheet 8 (dokumentasi jobsheet-08 §6)
Update	Mengubah data yang sudah ada	UPDATE	Jobsheet 9 (bab 2)
Delete	Menghapus data	DELETE	Jobsheet 9 (bab 3)

Hampir semua aplikasi berbasis data — dari SIMPUS-Mini sampai aplikasi sekelas media sosial atau e-commerce — pada dasarnya menjalankan

kombinasi 4 operasi ini berulang-ulang. Menguasai pola CRUD berarti menguasai fondasi **mayoritas** aplikasi web yang pernah kamu pakai.

1.2 Pola yang Berulang: Form Kosong vs Form Terisi

Perhatikan **Create** ([dokumentasi jobsheet-08](#))

dan **Update** ([bab 2](#)) sebenarnya memakai

pola form yang sangat mirip — bedanya:

	Create (tambah.php)	Update (edit.php)
Kondisi awal form	Kosong , siap diisi dari nol	Sudah terisi data yang mau diubah
Query yang dijalankan setelah submit	INSERT (baris baru)	UPDATE (baris yang sudah ada)
Perlu tahu identitas baris?	Tidak — baris ini belum ada	Ya — perlu tahu baris mana yang diubah (lewat id)

Poin terakhir ini penting: UPDATE dan DELETE **selalu** butuh cara untuk menunjuk **satu baris spesifik** — di sinilah kolom **id** (`SERIAL PRIMARY KEY`, ingat dari

[dokumentasi jobsheet-08 §2.4](#))

akhirnya benar-benar dipakai untuk pertama kalinya, setelah sejak jobsheet-08 hanya "ikut ter-fetch" tanpa dipakai di tampilan (ingat catatan ini dari

[dokumentasi jobsheet-08 README](#)).

1.3 Kenapa Delete Butuh Perhatian Ekstra?

Di antara 4 operasi CRUD, **Delete** adalah satu-satunya yang **merusak** (destruktif) — sekali data dihapus, **tidak ada cara** "undo" bawaan (beda dengan Update yang salah, masih bisa diubah lagi lewat form Edit). Karena risikonya, jobsheet ini menerapkan **pengaman tambahan** yang tidak ada di operasi lain: memastikan Delete **hanya** bisa dipicu lewat `method="post"` (bukan GET, seperti tautan biasa) — dijelaskan alasannya secara mendalam di [bab 3](#).

1.4 Peta Lengkap: Semua Fitur di Setiap Tabel

buku/	
└─ list.php	→ Read (+ pagination & pencarian, bab 5)
└─ tambah.php + proses_tambah.php	→ Create (sejak jobsheet-08)
└─ edit.php + proses_edit.php	→ Update (BARU)
└─ hapus.php	→ Delete (BARU)

Struktur yang **persis sama** berlaku untuk folder `anggota/`. Dengan peta ini, kamu siap membedah tiap file baru mulai bab 2.

2. Mengubah Data: `edit.php` & `proses_edit.php`

Dua file baru yang bekerja sama untuk fitur **Update** — mirip pola `tambah.php` + `proses_tambah.php` yang sudah kamu kenal, tapi dengan satu langkah tambahan penting: **mengambil data lama terlebih dulu**.

2.1 `buku/edit.php` — Kode Lengkap

```
<?php
$page_title = "Edit Buku";
include __DIR__ . '/../includes/header.php';
require __DIR__ . '/../includes/koneksi.php';
$flash = $_SESSION['flash'] ?? null;
unset($_SESSION['flash']);
$id = $_GET['id'] ?? null;
if (!$id) {
    header('Location: list.php');
    exit;
}
$stmt = $pdo->prepare("SELECT * FROM buku WHERE id = :id");
```

```

$stmt->execute(['id' => $id]);
$buku = $stmt->fetch(PDO::FETCH_ASSOC);
if (!$buku) {
    header('Location: list.php');
    exit;
}
?>
<section>
    <h2>Edit Buku</h2>
    ...
    <form id="form-tambah" method="post"
action="proses_edit.php">
        <input type="hidden" name="id" value="<?php echo
$buku['id']; ?>">
        <p>
            <label for="judul">Judul</label><br>
            <input type="text" id="judul" name="judul"
value="<?php echo $buku['judul']; ?>" required>
        </p>
        ...
    </form>
</section>

```

2.2 Mengambil id dari URL: \$_GET['id']

```

$id = $_GET['id'] ?? null;
if (!$id) {
    header('Location: list.php');
    exit;
}

```

- `\$\_GET` adalah superglobal baru (ingat dari

[dokumentasi jobsheet-07 §1.5](#)) —

bedanya dengan `$_POST`, `$_GET` mengambil data yang ditempelkan ke URL itu sendiri, dalam bentuk `?nama=nilai`. Ingat dari

[dokumentasi jobsheet-01 §4.2](#):

ini seperti "kebalikan" dari `method="post"` — datanya **terlihat** di address bar.

- Halaman ini dibuka lewat tautan seperti

`edit.php?id=3` — angka 3 di situ adalah `id` buku yang mau diubah,

dan `$_GET['id']` mengambilnya. Ingat dari mana tautan ini berasal:

```
<a href="edit.php?id=<?php echo $buku['id']; ?>" class="btn-edit">Edit</a>
```

di `list.php`, dibahas di [bab 5](#).

- Kalau `$id` kosong (halaman ini dibuka tanpa parameter `id` sama sekali, misalnya diketik manual tanpa `?id=...`), pengguna

diarahkan kembali ke `list.php` — mencegah kode di bawahnya mencoba mencari buku dengan `id` yang tidak jelas.

2.3 Mengambil Data Lama: `SELECT ... WHERE id = :id`

```
$stmt = $pdo->prepare("SELECT * FROM buku WHERE id = :id");
$stmt->execute(['id' => $id]);
$buku = $stmt->fetch(PDO::FETCH_ASSOC);
if (!$buku) {
    header('Location: list.php');
    exit;
}
```

- ``WHERE id = :id`` — klausa SQL baru yang belum pernah dipakai di jobsheet-08: `WHERE` **menyaring** baris mana saja yang diambil (tanpa `WHERE`, `SELECT` mengambil **semua** baris, ingat dari [dokumentasi jobsheet-08 §6.1](#)) — di sini hanya baris yang `id`-nya cocok dengan `:id`. Ini prepared statement, sama konsepnya dengan `INSERT` di [dokumentasi jobsheet-08 §5](#) — placeholder `:id` diisi nilai sungguhan lewat `execute(['id' => $id])`.
- ``->fetch(PDO::FETCH_ASSOC)`` — method baru, **tunggal**, beda dari `fetchAll()` yang sudah kamu kenal ([dokumentasi jobsheet-08 §6.3](#)): `fetch()` mengambil **satu baris saja** (masuk akal di sini — karena `id` adalah `PRIMARY KEY`, ingat dari [dokumentasi jobsheet-08 §2.4](#), query `WHERE id = :id` **paling banyak** hanya akan cocok dengan satu baris).
- Kalau `$buku` ternyata `false`/kosong (misalnya `id` yang diminta tidak ada di database — sudah dihapus, atau memang tidak pernah ada), pengguna diarahkan kembali ke `list.php` — mencegah halaman menampilkan form dengan data yang tidak ada.

2.4 Mengisi Form dengan Data Lama: Atribut value

```
<input type="text" id="judul" name="judul" value="<?php echo $buku['judul']; ?>" required>
```

Ini **perbedaan visual** paling mencolok dibanding form Tambah Buku

(`tambah.php`, ingat dari

[dokumentasi jobsheet-01 §4.1](#)):

atribut ``value="..."`` diisi dengan data yang **sudah ada**

(`$buku['judul']`), sehingga saat halaman dibuka, kotak input sudah

terisi teks yang mau diubah — pengguna tidak perlu mengetik ulang

semuanya dari nol, cukup mengubah bagian yang perlu diperbaiki.

Kasus khusus untuk `<select>`:

```
<select id="kategori" name="kategori">
  <?php foreach (['fiksi' => 'Fiksi', 'non-fiksi' => 'Non-Fiksi',
    'referensi' => 'Referensi'] as $value => $label): ?>
    <option value="<?php echo $value; ?>" <?php echo $buku['kategori']
    === $value ? 'selected' : ''; ?>><?php echo $label; ?></option>
  <?php endforeach; ?>
</select>
```

- ``foreach ([...] as $value => $label)`` — mengulang array asosiatif yang berisi 3 pasangan (nilai teknis → label tampilan), menghasilkan 3 elemen `<option>` — cara yang lebih ringkas dibanding menulis 3 baris `<option>` manual seperti di

[dokumentasi jobsheet-01](#).

- ``$buku['kategori'] === $value ? 'selected' : ''`` — untuk **setiap** opsi, memeriksa apakah nilainya **cocok** dengan kategori buku yang sedang diedit. Kalau cocok, atribut HTML ``selected`` ditambahkan — inilah cara memberi tahu `<select>` opsi mana yang harus tampil

terpilih sejak awal, konsep yang setara dengan `value="..."` pada `<input>` biasa, hanya sedikit berbeda mekanismenya karena `<select>` bekerja dengan daftar opsi, bukan satu kotak teks tunggal.

2.5 Kotak Tersembunyi: `<input type="hidden" name="id">`

```
<input type="hidden" name="id" value="<?php echo $buku['id']; ?>">
```

`type="hidden"` adalah jenis input baru — sebuah field yang tidak tampil sama sekali **di layar, tapi** tetap ikut terkirim bersama form saat di-submit (persis seperti field biasa lainnya). Ini dipakai untuk membawa informasi yang **dibutuhkan server** (id buku yang sedang diedit) tapi **tidak perlu dilihat atau diubah** pengguna — beda dari `judul/pengarang/dst.` yang memang perlu diketik ulang pengguna. Tanpa field tersembunyi ini, `proses_edit.php` ([§2.6](#)) tidak akan tahu **baris mana** yang harus di-UPDATE.

2.6 `proses_edit.php`: Menjalankan UPDATE

```
$id = $_POST['id'] ?? null;
```

```
// ...ambil field lain dari $_POST, validasi (identik dengan
proses_tambah.php)...
if (!$id) {
    header('Location: list.php');
    exit;
}
// ...kalau ada error validasi, redirect ke edit.php?id=... (bukan
tambah.php)...
$stmt = $pdo->prepare(
    "UPDATE buku SET judul = :judul, pengarang = :pengarang, tahun =
:tahun,
    isbn = :isbn, stok = :stok, kategori = :kategori WHERE id = :id"
);
$stmt->execute([
    'judul' => $judul,
    'pengarang' => $pengarang,
    'tahun' => (int) $tahun,
    'isbn' => $isbn,
    'stok' => (int) $stok,
    'kategori' => $kategori,
    'id' => $id,
]);
```

- ``$id = $_POST['id'] ?? null;`` — sekarang mengambil id dari ``$_POST``, bukan `$_GET` seperti di `edit.php` ([§2.2](#)) — karena field tersembunyi di [§2.5](#) dikirim lewat `method="post"` (ingat dari [dokumentasi jobsheet-07 §4.1](#)), bukan lagi lewat URL.

- **Validasi tetap sama persis** dengan `proses_tambah.php` ([dokumentasi jobsheet-07 §4.3](#)) —

aturan yang sama berlaku baik saat menambah data baru **maupun** mengubah data lama. Satu-satunya beda: kalau validasi gagal, redirect mengarah ke ``edit.php?id=' . urlencode($id)``, bukan `tambah.php` — mengembalikan pengguna ke form edit yang sama (bukan form kosong).

- ``UPDATE buku SET kolom1 = :kolom1, ... WHERE id = :id`` — perintah

SQL baru: `SET` menentukan kolom mana yang **nilainya diubah**, dan ``WHERE id = :id`` memastikan perubahan **hanya** memengaruhi satu baris spesifik. **Ini bagian paling krusial untuk dipahami:** kalau klausa `WHERE` ini **terlupa/hilang**, perintah `UPDATE` akan **mengubah SEMUA baris** di tabel `buku` sekaligus dengan nilai yang sama — kesalahan serius yang bisa merusak seluruh data tabel dalam sekali jalan. Selalu periksa klausa `WHERE` ada dan benar setiap kali menulis `UPDATE/DELETE`.

- ``urlencode($id)`` — fungsi PHP yang mengubah karakter tertentu supaya **aman** dimasukkan ke dalam URL (misalnya spasi diubah jadi `%20`). Meski `id` di sini biasanya berupa angka biasa (tidak mengandung karakter spesial), ini kebiasaan baik yang aman diterapkan kapan pun sebuah nilai disisipkan ke URL.

3. Menghapus Data: `hapus.php`

File paling singkat di jobsheet ini, tapi menyimpan detail keamanan paling penting untuk dipahami.

3.1 Kode Lengkap

```
<?php
session_start();
require __DIR__ . '/../includes/koneksi.php';
// Sengaja hanya menerima POST (bukan GET) agar penghapusan tidak bisa
// dipicu tanpa sengaja lewat link/preview crawler.
if ($_SERVER['REQUEST_METHOD'] !== 'POST') {
    header('Location: list.php');
    exit;
}
$id = $_POST['id'] ?? null;
if ($id) {
    $stmt = $pdo->prepare("DELETE FROM buku WHERE id = :id");
    $stmt->execute(['id' => $id]);
    $_SESSION['flash'] = ['type' => 'success', 'pesan' => 'Buku
berhasil dihapus.'];
}
header('Location: list.php');
exit;
```

3.2 Kenapa Delete Tidak Boleh Sesederhana Sebuah Tautan?

Bayangkan seandainya menghapus buku dibuat **semudah** tautan biasa:

```
<a href="hapus.php?id=3">Hapus</a>
```

Ini terlihat sederhana, tapi punya bahaya tersembunyi: tautan

`<a href>` selalu memakai GET — dan permintaan GET bisa terpicu

tanpa sengaja oleh banyak hal di luar kendali pengguna:

- **Crawler mesin pencari** (seperti Googlebot) yang menjelajahi setiap tautan di halaman untuk mengindeksnya — kalau tautan Hapus ada di halaman publik, crawler bisa "mengklik"-nya begitu saja saat menjelajah.
- **Fitur pratinjau tautan** di beberapa aplikasi chat/email yang "mengunjungi" URL secara otomatis untuk menampilkan pratinjau, tanpa pengguna benar-benar mengklik sama sekali.
- **Tombol "back"** browser yang kadang mengulang permintaan terakhir.

Karena alasan-alasan ini, ada konvensi penting dalam pengembangan web:

GET seharusnya hanya dipakai untuk operasi yang aman diulang tanpa efek samping (seperti menampilkan halaman, ingat

`buku/list.php?page=2` di [bab 5](#)),

sedangkan operasi yang **mengubah data** (apalagi yang **merusak** seperti Delete) harus memakai POST.

3.3 Memeriksa Metode HTTP:

`$_SERVER['REQUEST_METHOD']`

```
if ($_SERVER['REQUEST_METHOD'] !== 'POST') {
```

```
header('Location: list.php');
exit;
}
```

- `\$\_SERVER` adalah superglobal PHP lain (ingat konsepnya dari [dokumentasi jobsheet-07 §1.5](#))

berisi informasi teknis tentang permintaan yang sedang diproses.

- `\$\_SERVER['REQUEST\_METHOD']` berisi teks 'GET' atau 'POST' (atau metode lain), tergantung **bagaimana** permintaan ke halaman ini dibuat.

- Kalau metodenya **bukan** 'POST' (misalnya seseorang mencoba membuka `hapus.php?id=3` langsung lewat address bar, yang selalu memakai GET), pengguna langsung **diarahkan kembali** ke `list.php` **tanpa** ada satu baris data pun yang terhapus — permintaan itu ditolak mentah-mentah sebelum sempat menyentuh database sama sekali.

3.4 Perintah DELETE

```
$id = $_POST['id'] ?? null;
if ($id) {
    $stmt = $pdo->prepare("DELETE FROM buku WHERE id = :id");
    $stmt->execute(['id' => $id]);
    $_SESSION['flash'] = ['type' => 'success', 'pesan' => 'Buku
    berhasil dihapus.'];
}
```

- ``DELETE FROM buku WHERE id = :id`` — perintah SQL baru: menghapus

baris dari tabel buku yang id-nya cocok. Sama seperti UPDATE

([dokumentasi jobsheet-09 §2.6](#)),

klausula `WHERE` di sini wajib ada — `DELETE FROM buku` **tanpa**

`WHERE` akan menghapus **SELURUH** baris di tabel buku sekaligus,

tanpa peringatan apa pun dari database.

- Perhatikan `$id` diperiksa dulu (`if ($id)`) sebelum query dijalankan — mencegah percobaan `DELETE ... WHERE id = :id` dengan `id` kosong/`null` yang bisa berperilaku tidak terduga.
- Kalau berhasil, flash message sukses diset — pola yang sama persis dengan Create ([dokumentasi jobsheet-08 §5.5](#)) dan Update ([bab 2 §2.6](#)).

3.5 Bagaimana Form HTML Memicu Ini? (Pratinjau Singkat)

Ingat dari [README.md](#) jobsheet ini, tombol Hapus di

`list.php` sekarang **bukan lagi** `<button>` polos, melainkan

`<form>` sungguhan:

```
<form class="form-hapus" method="post" action="hapus.php">
  <input type="hidden" name="id" value="<?php echo $buku['id']; ?>">
  <button type="submit" class="btn-hapus">Hapus</button>
</form>
```

- `method="post"` — inilah yang membuat permintaan ini **selalu** memakai POST, lolos pemeriksaan di [§3.3](#).
- `<input type="hidden" name="id">` — pola yang **sama persis** dengan form Edit ([bab 2 §2.5](#)) — membawa id baris yang mau dihapus tanpa perlu ditampilkan ke pengguna.
- `<button type="submit">` — ingat dari [dokumentasi jobsheet-01 §4.5](#), tombol ini yang memicu form untuk benar-benar di-submit ke `action="hapus.php"`.

Perubahan konfirmasi lewat JavaScript untuk pola `<form>` baru ini dibahas detail di [bab 4](#).

4. JS: Konfirmasi Hapus via Event submit

Karena tombol Hapus sekarang berupa `<form>` sungguhan (ingat dari [bab 3 §3.5](#)), `initHapusConfirm` di `app.js` perlu diubah cara kerjanya secara mendasar.

4.1 Kode Sebelum dan Sesudah

Sebelumnya (jobsheet-06, event delegation pada `click`):

```
function initHapusConfirm() {
    document.addEventListener("click", function (e) {
        const btn = e.target.closest(".btn-hapus");
        if (!btn) return;
        const row = btn.closest("tr");
        const nama = row ? row.querySelector("td")?.textContent : "data
        ini";
        const yakin = confirm("Yakin ingin menghapus \"" + nama +
        "\"?");
        if (yakin && row) {
            row.remove();
        }
    });
}
```

Sekarang (jobsheet-09, event delegation pada `submit`):

```
// Tombol Hapus kini berada di dalam <form class="form-hapus"
method="post">
// yang benar-benar mengirim request DELETE ke server (buku/hapus.php,
// anggota/hapus.php). Konfirmasi dilakukan pada event "submit" agar
bisa
// dibatalkan (preventDefault) sebelum request terkirim.
function initHapusConfirm() {
    document.addEventListener("submit", function (e) {
        const form = e.target;
        if (!form.classList.contains("form-hapus")) return;
        const row = form.closest("tr");
        const nama = row ? row.querySelector("td")?.textContent : "data
        ini";
        const yakin = confirm("Yakin ingin menghapus \"" + nama +
        "\"?");
```

```

        if (!yakin) {
            e.preventDefault();
        }
    });
}

```

4.2 Kenapa Tidak Bisa Memakai Pola click Lagi?

Ingat dari [dokumentasi jobsheet-05](#)

dan [dokumentasi jobsheet-06 §6](#),

versi lama bekerja dengan cara: tunggu klik pada tombol, tampilkan

`confirm()`, **kalau OK** baru hapus barisnya dari tampilan lewat

`row.remove()`. Pola ini masuk akal ketika tombol Hapus tidak

benar-benar mengirim apa pun ke server — menghapus baris dari DOM

sudah cukup (ingat catatan "front-end only, belum ke server" dari

[dokumentasi jobsheet-05 §5.6](#)).

Sekarang tombol Hapus berada di dalam `<form method="post" action="hapus.php">`

yang **sungguhan** mengirim data ke server ([bab 3](#)).

Kalau kode JS tetap menunggu event `click` pada tombol lalu baru

menampilkan `confirm()`, form akan tetap ter-submit ke server lebih

dulu (klik tombol submit memicu submit form secara otomatis,

terlepas dari apa pun yang terjadi di event listener `click` yang

terpisah) — pesan konfirmasi jadi percuma karena data sudah kadung

terkirim dan buku sudah terhapus sungguhan sebelum pengguna sempat

membatalkannya.

4.3 Solusi: Dengarkan Event submit, Bukan click

```
document.addEventListener("submit", function (e) {
  const form = e.target;
  if (!form.classList.contains("form-hapus")) return;
  ...
});
```

- Event delegation di `document` — pola yang sama dari [dokumentasi jobsheet-06 §6.3](#), hanya sekarang mendengarkan event `submit`, bukan `click`. Event `submit` terjadi tepat **sebelum** browser benar-benar mengirim data form — memberi kesempatan JavaScript untuk **ikut campur** sebelum pengiriman sungguhan terjadi.

- `e.target` di sini adalah `<form>` itu sendiri (karena event `submit` terjadi pada elemen `<form>`, bukan pada tombolnya) — beda dari `e.target` pada event `click` di [jobsheet-06](#) yang merujuk ke elemen paling spesifik yang diklik.

- `form.classList.contains("form-hapus")` — memeriksa apakah form yang di-submit ini memang form Hapus (ingat class `form-hapus` dari [bab 3 §3.5](#)), bukan form lain di halaman yang sama (seperti form pencarian dari [bab 5](#), atau form Tambah/Edit yang punya validasinya sendiri di

[dokumentasi jobsheet-05 §7](#)).

Kalau bukan, `return` — biarkan form itu diproses seperti biasa, tanpa campur tangan fungsi ini.

4.4 Membatalkan Submit dengan `preventDefault()`

```
const yakin = confirm("Yakin ingin menghapus \"" + nama + "\"?");
if (!yakin) {
  e.preventDefault();
}
```

Ini **kebalikan logika** dibanding versi jobsheet-06: dulu, aksi `(row.remove())` dijalankan **kalau** pengguna menekan OK. Sekarang, karena form akan **otomatis** ter-submit ke server kecuali dicegah, logikanya dibalik: `e.preventDefault()` (ingat method ini dari [dokumentasi jobsheet-05 §7.7](#)) dipanggil **kalau pengguna menekan Cancel** (`!yakin`, artinya `yakin` bernilai `false`) — **membatalkan** pengiriman form sepenuhnya. Kalau pengguna menekan OK, **tidak ada** yang dicegah — form melanjutkan proses submit normalnya ke `hapus.php`, benar-benar menghapus data di database ([bab 3](#)).

4.5 Mencari Baris: `form.closest("tr")`

```
const row = form.closest("tr");
```

Pola `.closest()` ini sudah kamu kenal dari

[dokumentasi jobsheet-05 §5.3](#) —

hanya sekarang dipanggil dari elemen `form` (bukan `btn` seperti sebelumnya), karena `<form class="form-hapus">` yang membungkus tombol sekarang **juga** berada langsung di dalam `<td>` yang sama, sejajar dengan `<tr>` yang sama pula (ingat struktur HTML dari [bab 3 §3.5](#)).

4.6 Perbandingan Ringkas Ketiga Versi `initHapusConfirm`

	Jobsheet-05	Jobsheet-06	Jobsheet-09
Event yang didengarkan	<code>click</code> (per tombol)	<code>click</code> (delegation di document)	<code>submit</code> (delegation di document)
Aksi kalau pengguna OK	<code>row.remove()</code> (hapus dari tampilan saja)	<code>row.remove()</code> (hapus dari tampilan saja)	Tidak ada — form lanjut submit ke <code>hapus.php</code> sungguhan
Aksi kalau pengguna Cancel	Tidak ada apa-apa	Tidak ada apa-apa	<code>e.preventDefault()</code> — batalkan submit

	Jobsheet-05	Jobsheet-06	Jobsheet-09
Data benar-benar terhapus?	Tidak (front-end only)	Tidak (front-end only)	Ya — lewat DELETE di database

Progresi ini menunjukkan bagaimana **pola JavaScript yang sama** (event delegation, `confirm()`, `.closest()`) terus dipakai ulang dan disesuaikan sedikit demi sedikit seiring aplikasi menjadi lebih sungguhan — bukan ditulis ulang dari nol setiap kali kebutuhannya berubah.

5. Pagination & Pencarian Sisi Server

Bab paling padat SQL di jobsheet ini — `buku/list.php` sekarang jauh lebih canggih dibanding versi jobsheet-08 yang sekadar `SELECT *`.

5.1 Kode Lengkap Bagian Query

```
$perPage = 5;
```

```

$page = max(1, (int) ($_GET['page'] ?? 1));
$offset = ($page - 1) * $perPage;
$keyword = trim($_GET['q'] ?? '');
if ($keyword !== '') {
    $hitung = $pdo->prepare("SELECT COUNT(*) FROM buku WHERE judul
    ILIKE :kw");
    $hitung->execute(['kw' => '%' . $keyword . '%']);
    $totalRows = $hitung->fetchColumn();
    $stmt = $pdo->prepare("SELECT * FROM buku WHERE judul ILIKE :kw
    ORDER BY id DESC LIMIT :limit OFFSET :offset");
    $stmt->bindValue('kw', '%' . $keyword . '%');
} else {
    $totalRows = $pdo->query("SELECT COUNT(*) FROM buku")-
    >fetchColumn();
    $stmt = $pdo->prepare("SELECT * FROM buku ORDER BY id DESC LIMIT
    :limit OFFSET :offset");
}
$stmt->bindValue('limit', $perPage, PDO::PARAM_INT);
$stmt->bindValue('offset', $offset, PDO::PARAM_INT);
$stmt->execute();
$daftarBuku = $stmt->fetchAll(PDO::FETCH_ASSOC);
$totalPages = max(1, (int) ceil($totalRows / $perPage));

```

5.2 Kenapa Butuh Pagination?

Bayangkan tabel `buku` sudah berisi 200 buku — menampilkan semuanya sekaligus dalam satu halaman akan membuat halaman sangat panjang dan lambat dimuat. **Pagination** (penomoran halaman) membagi data jadi beberapa "halaman" berisi sejumlah baris tetap — di jobsheet ini, **5 baris per halaman** (`$perPage = 5`).

5.3 Menghitung Halaman: \$page dan \$offset

```
$page = max(1, (int) ($_GET['page'] ?? 1));
$offset = ($page - 1) * $perPage;
```

- ``$_GET['page'] ?? 1`` — nomor halaman diambil dari URL (ingat `$_GET` dari [dokumentasi jobsheet-09 §2.2](#)),

misalnya dari `list.php?page=2`. Kalau parameter `page` tidak ada sama sekali (pertama kali membuka halaman), dianggap halaman 1.

- ``max(1, ...)`` — fungsi PHP yang mengembalikan nilai **terbesar** di antara argumennya. Dipakai di sini sebagai **pengaman**: kalau seseorang mencoba membuka `list.php?page=0` atau `page=-5` (nilai yang tidak masuk akal), `$page` akan tetap dipaksa minimal 1.

- ``$offset = ($page - 1) * $perPage;`` — **offset** adalah "berapa baris yang perlu dilompati dari awal" sebelum mulai mengambil data.

Kalau `$page = 1`: offset $(1-1) * 5 = 0$ (mulai dari baris pertama). Kalau `$page = 2`: offset $(2-1) * 5 = 5$ (lompati 5 baris pertama, mulai dari baris ke-6). Kalau `$page = 3`: offset 10, dan seterusnya.

5.4 LIMIT dan OFFSET dalam Query SQL

```
SELECT * FROM buku ORDER BY id DESC LIMIT :limit OFFSET :offset
```

- ``LIMIT :limit`` — klausa SQL baru: membatasi hasil **maksimal**

sejumlah baris tertentu (di sini, `$perPage = 5`).

- ``OFFSET :offset`` — melompati sejumlah baris dari awal sebelum mulai mengambil (dihitung di [§5.3](#)).

Kombinasi keduanya inilah yang menghasilkan "potongan" data per halaman: halaman 1 mengambil baris ke 1-5, halaman 2 mengambil baris ke 6-10, dan seterusnya — ``ORDER BY id DESC`` (ingat dari [dokumentasi jobsheet-08 §6.1](#))

tetap penting di sini supaya urutan potongannya **konsisten** setiap kali — tanpa `ORDER BY`, database **tidak menjamin** urutan baris yang sama setiap query, yang bisa membuat pagination menampilkan data duplikat atau melewati baris tertentu.

5.5 `bindValue()` dan `PDO::PARAM_INT`

```
$stmt->bindValue('limit', $perPage, PDO::PARAM_INT);
$stmt->bindValue('offset', $offset, PDO::PARAM_INT);
$stmt->execute();
```

- ``bindValue(nama, nilai, tipe)`` — cara **lain** mengisi placeholder prepared statement, selain lewat array di `execute([...])` seperti yang sudah kamu kenal dari [dokumentasi jobsheet-08 §5.4](#).

`bindValue()` mengisi placeholder **satu per satu**, dipanggil **sebelum** `execute()` (yang di sini dipanggil **tanpa** argumen

sama sekali, karena semua placeholder sudah diisi lebih dulu).

- ``PDO::PARAM_INT`` — parameter ketiga yang **secara eksplisit** memberi tahu PDO bahwa nilai ini harus diperlakukan sebagai **angka**, bukan teks. Ini penting khusus untuk `LIMIT/OFFSET` — PostgreSQL cukup ketat soal tipe data di klausa ini, dan tanpa `PDO::PARAM_INT`, nilai yang terkirim sebagai teks (perilaku default `bindValue/execute` biasa) bisa menyebabkan error SQL.

5.6 Pencarian Sisi Server: ILIKE

```
if ($keyword !== '') {
    $hitung = $pdo->prepare("SELECT COUNT(*) FROM buku WHERE judul
    ILIKE :kw");
    $hitung->execute(['kw' => '%' . $keyword . '%']);
    $totalRows = $hitung->fetchColumn();
    $stmt = $pdo->prepare("SELECT * FROM buku WHERE judul ILIKE :kw
    ORDER BY id DESC LIMIT :limit OFFSET :offset");
    $stmt->bindValue('kw', '%' . $keyword . '%');
}
```

- ``$keyword = trim($_GET['q'] ?? '')`` — kata kunci pencarian diambil dari parameter URL `q` (dari form `method="get"`, dibahas di [§5.8](#)).

- ``ILIKE`` — operator pencocokan teks khusus PostgreSQL, mirip `LIKE` biasa tapi **tidak peduli huruf besar/kecil** (*case-insensitive*) — konsepnya setara dengan `.toLowerCase()` yang sudah

kamu pakai untuk pencarian client-side sejak

[dokumentasi jobsheet-05 §6.4](#).

- ``%'. $keyword . '%`` — tanda persen (%) di kedua sisi kata kunci adalah **wildcard** (karakter pengganti bebas) di SQL, berarti "boleh ada teks apa saja sebelum dan sesudah kata kunci ini." Kalau `$keyword` adalah "bumi", pola `'%bumi%'` akan cocok dengan judul apa pun yang **mengandung** kata "bumi" di bagian mana pun — persis konsepnya dengan `teks.includes(keyword)` yang sudah kamu pakai di JavaScript

([dokumentasi jobsheet-05 §6.5](#)).

- Perhatikan `$totalRows` **juga** dihitung ulang dengan `WHERE/ILIKE` yang sama — ini penting supaya jumlah halaman ([§5.7](#)) dihitung berdasarkan **jumlah hasil pencarian**, bukan jumlah total seluruh buku di database.

5.7 Menghitung Total Halaman

```
$totalPages = max(1, (int) ceil($totalRows / $perPage));
```

- ``ceil(...)`` — fungsi pembulatan **ke atas**. Kalau ada 12 buku dengan 5 per halaman, `12 / 5 = 2.4`, dibulatkan ke atas jadi 3 halaman (2 halaman penuh + 1 halaman berisi sisa 2 buku).
- ``max(1, ...)`` — pengaman lagi: kalau `$totalRows` adalah 0 (tidak ada data/hasil pencarian sama sekali), `$totalPages` tetap dipaksa minimal 1 — supaya navigasi halaman tetap menampilkan

minimal satu tombol "1", bukan kosong sama sekali.

5.8 Form Pencarian: method="get"

```
<form method="get" action="list.php">
  <span>
    <label for="search-input">Cari Judul Buku</label><br>
    <input type="text" id="search-input" name="q" value="<?php echo
$keyword; ?>" placeholder="Ketik judul buku...">
  </span>
  <button type="submit">Cari</button>
</form>
```

Berbeda dari form Tambah/Edit yang memakai `method="post"` (ingat dari [dokumentasi jobsheet-07 §4.1](#)),

form pencarian ini sengaja memakai `method="get"` — cocok karena mencari data **tidak mengubah apa pun** di server (aman diulang berkali-kali, aman jadi tautan yang bisa di-bookmark/dibagikan), sesuai prinsip yang sudah dibahas di

[bab 3 §3.2](#) —

GET untuk operasi aman/dibaca, POST untuk operasi yang mengubah data.

`value="<?php echo $keyword; ?>"` membuat kata kunci yang baru saja dicari **tetap terlihat** di kotak input setelah halaman submit — tanpa baris ini, kotak pencarian akan **kosong lagi** setiap kali halaman dimuat ulang, meskipun pencariannya berhasil dan hasilnya tampil. Ingat catatan penting dari [README.md](#) jobsheet

ini: nilai ini **belum di-escape** sebelum ditampilkan kembali — celah

ini akan dibahas dan diperbaiki di Jobsheet 11.

5.9 Menampilkan Navigasi Halaman

```
<nav class="pagination">
  <?php for ($i = 1; $i <= $totalPages; $i++): ?>
    <a href="list.php?page=<?php echo $i; ?><?php echo $keyword !== ''
? '&q=' . urlencode($keyword) : ''; ?>"
    class="<?php echo $i === $page ? 'active' : ''; ?>"><?php echo
$i; ?></a>
  <?php endfor; ?>
</nav>
```

- ``for ($i = 1; $i <= $totalPages; $i++)`` — perulangan PHP baru (belum pernah dipakai sebelumnya di dokumentasi ini): "mulai dari `i=1`, ulangi selama `i` masih kurang dari atau sama dengan `$totalPages`, tambah `i` satu setiap putaran" — menghasilkan satu tautan angka untuk **setiap** nomor halaman yang ada.
- ``&q=' . urlencode($keyword)`` ditambahkan **hanya kalau** sedang ada pencarian aktif (`$keyword !== ''`) — memastikan kata kunci pencarian **ikut terbawa** saat berpindah antar halaman hasil pencarian, bukan hilang begitu saja.
- ``class="<?php echo $i === $page ? 'active' : ''; ?>"`` — halaman yang **sedang aktif** diberi class `active`, dipakai CSS untuk menyorotnya secara visual (dibahas di [bab 6](#)).

6. CSS Pendukung Fitur Baru

Beberapa penambahan `style.css` untuk mendukung tautan Edit, form Hapus, pagination, dan form pencarian yang sudah dibahas di bab-bab sebelumnya.

6.1 Tautan Edit yang Terlihat Seperti Tombol

```
td button:first-of-type,
td a.btn-edit {
  background-color: #f0ad4e;
  color: #fff;
}
td a.btn-edit {
  display: inline-block;
  padding: 0.35rem 0.7rem;
  margin-right: 0.35rem;
  border-radius: 4px;
  font-size: 0.85rem;
}
```

Ingat dari [bab 2](#), tombol Edit sekarang berupa

tag `<a>` (tautan biasa, ke `edit.php?id=...`), **bukan**

`<button>` seperti versi jobsheet-05/06/07/08 (yang belum benar-benar

berfungsi). Masalahnya: `<a>` secara default **tidak terlihat** seperti

tombol — hanya teks biru bergaris bawah (ingat gaya dasar `<a>` dari

[dokumentasi jobsheet-02 §3.3](#)).

Aturan `td a.btn-edit` di sini "menyamarkan" tautan itu supaya terlihat **identik** dengan tombol Hapus di sebelahnya:

- `display: inline-block` — wajib diberikan, karena `<a>` adalah elemen *inline* secara default (ingat konsep ini dari [dokumentasi jobsheet-02 §8.3](#)) —
- tanpa ini, properti `padding` vertikal pada tautan tidak akan memberi ruang yang terlihat seperti tombol sungguhan.
- Sisa propertinya (`padding`, `border-radius`, `font-size`) **disamakan** dengan gaya `td button` yang sudah ada sejak [dokumentasi jobsheet-02 §7.6](#).

- Warna oranyenya (`#f0ad4e`) digabung lewat selector `td button:first-of-type, td a.btn-edit` —
- dua selector berbeda (satu untuk kasus lama, satu untuk kasus baru) yang **kebetulan** butuh warna sama, digabung jadi satu aturan dengan koma (pola yang sama dari [dokumentasi jobsheet-02 §7.3](#)).

6.2 Form Hapus yang Tidak Merusak Tata Letak

```
td form.form-hapus {
  display: inline;
}
```

Ingat dari [bab 3](#), tombol Hapus sekarang

dibungkus `<form>`. Masalahnya: `<form>` secara default adalah elemen **block** (memenuhi satu baris penuh, mendorong elemen setelahnya turun ke baris baru) — kalau dibiarkan begitu, tombol Edit dan form Hapus yang seharusnya **sejajar** di satu sel `<td>` akan malah terpisah baris. `display: inline` memaksa `<form>` ini berperilaku seperti elemen *inline* biasa (mengalir sejajar dengan konten di sekitarnya, seperti teks biasa), sehingga tautan Edit dan tombol Hapus tetap terlihat berdampingan seperti sebelumnya.

6.3 Navigasi Pagination

```
.pagination {
  display: flex;
  gap: 0.5rem;
  margin-top: 1rem;
}

.pagination a {
  padding: 0.4rem 0.75rem;
  border: 1px solid #cdd4da;
  border-radius: 4px;
  color: #1d5b8a;
}

.pagination a.active {
  background-color: #1d5b8a;
  color: #fff;
  border-color: #1d5b8a;
}
```

- `.pagination { display: flex; gap: 0.5rem; }` — ingat Flexbox dari [dokumentasi jobsheet-02 §4](#):

menyusun semua tautan nomor halaman ([bab 5 §5.9](#))

sejajar horizontal dengan jarak seragam.

- `.pagination a` — setiap nomor halaman diberi bingkai tipis dan sudut membulat, terlihat seperti tombol kecil yang bisa diklik.

- `.pagination a.active` — selector gabungan (elemen `<a>` yang juga punya class `active`) memberi warna latar biru tema penuh

untuk halaman yang **sedang dibuka** — ingat class `active` ini

ditambahkan lewat PHP di [bab 5 §5.9](#)

`($i === $page ? 'active' : '')`, memberi umpan balik visual jelas

"kamu sedang di halaman ini."

6.4 Form Pencarian Sejajar dengan Tombol "Cari"

```
.search-box form {
  display: flex;
  gap: 0.5rem;
  align-items: flex-end;
}
.search-box button {
  padding: 0.55rem 1.2rem;
  border: none;
  border-radius: 4px;
  background-color: #1d5b8a;
  color: #fff;
```

```

    cursor: pointer;
}

```

- **`display: flex; align-items: flex-end;`** — menyusun label+input pencarian ([bab 5 §5.8](#))

sejajar horizontal dengan tombol "Cari" di sampingnya.

`align-items: flex-end` (belum pernah dipakai sebelumnya di dokumentasi ini) menyejajarkan kedua elemen berdasarkan tepi bawahnya — penting di sini karena `<span>` pembungkus label+input lebih tinggi (2 baris: label lalu input) dibanding tombol "Cari" (1 baris saja), dan tanpa pengaturan ini keduanya akan terlihat tidak sejajar rapi secara vertikal.

- **`.search-box button`** — gaya tombol "Cari" dibuat senada dengan tombol submit form lain ([dokumentasi jobsheet-02 §8.5](#)), warna biru tema solid dengan teks putih.

7. Rangkuman & Latihan Lanjutan

7.1 Rangkuman Keseluruhan Jobsheet 9

Bagian	Konsep yang Dipelajari
Konsep Dasar CRUD	Create/Read/Update/Delete, kenapa Update/Delete butuh id
Edit & Update	<code>\$_GET['id']</code> , <code>SELECT ... WHERE</code> , <code>fetch()</code> tunggal, <code>value="..."</code> , <code>UPDATE ... SET ... WHERE</code>
Hapus & Delete	Kenapa Delete harus POST, <code>\$_SERVER['REQUEST_METHOD']</code> , <code>DELETE FROM ... WHERE</code>
JS Konfirmasi Hapus	Event submit vs click, <code>preventDefault()</code> untuk membatalkan submit sungguhan
Pagination & Pencarian Server	<code>LIMIT/OFFSET</code> , <code>ILIKE</code> , <code>bindValue()</code> , <code>PDO::PARAM_INT</code> , <code>method="get"</code>
CSS Pendukung	Menyamarkan <code><a></code> jadi tombol, <code>display: inline</code> pada form, styling pagination

7.2 Konsep Inti yang Perlu Diingat

12. **`UPDATE`/`DELETE` tanpa `WHERE` sangat berbahaya** — selalu memastikan klausa `WHERE id = :id` ada sebelum menjalankan kedua perintah ini, kalau tidak seluruh tabel bisa berubah/terhapus sekaligus

([bab 2 §2.6](#), [bab 3 §3.4](#)).

13. GET untuk operasi aman, POST untuk operasi yang mengubah data — terutama untuk operasi destruktif seperti Delete, yang sengaja diblokir kalau diakses lewat GET ([bab 3 §3.2-3.3](#)).

14. Event ``submit`` bisa dibatalkan sebelum data terkirim, beda dari menunggu click lalu bereaksi setelahnya — penting begitu form benar-benar terhubung ke aksi sungguhan di server ([bab 4](#)).

15. Pagination (LIMIT/OFFSET) mencegah satu halaman menampilkan seluruh data sekaligus, penting untuk performa begitu jumlah data bertambah banyak ([bab 5 §5.2-5.4](#)).

16. Pencarian sisi server (``ILIKE``) berbeda tujuannya dari filter sisi klien yang sudah kamu bangun sejak jobsheet-05 — server bisa mencari **lintas semua halaman/data**, klien hanya menyaring apa yang **sedang tampil** ([bab 5 §5.6](#)).

7.3 Cara Mencoba Sendiri

17. Jalankan `php -S localhost:8000` atau lewat Laragon (pastikan database dari [dokumentasi jobsheet-08](#) sudah siap), buka `http://localhost:8000/index.php`.

18. Ikuti siklus lengkap dari [README.md](#) jobsheet ini: **tambah → tampil → ubah (Edit) → tampil berubah →**

hapus → hilang dari list.

19. Tambahkan **lebih dari 5 buku** (misalnya 7-8 buku), lalu perhatikan navigasi pagination muncul dengan lebih dari satu nomor halaman — klik ke halaman 2, amati 5 buku pertama (yang paling baru ditambahkan, ingat ORDER BY id DESC) tetap di halaman 1.

20. Ketik kata kunci di kolom pencarian, klik "Cari" — amati URL berubah jadi `list.php?q=...`, dan pagination (kalau hasilnya lebih dari 5) ikut menyesuaikan dengan jumlah hasil pencarian, bukan jumlah total buku.

21. Coba klik tombol Hapus, lalu klik **Cancel** di dialog konfirmasi — amati baris **tidak** hilang dan **tidak** ada permintaan yang terkirim ke server (buka DevTools tab **Network** untuk membuktikannya).

22. Coba buka `buku/hapus.php` **langsung** lewat address bar (tanpa `?id=...` apa pun, dan tanpa lewat form) — amati kamu langsung diarahkan ke `list.php` **tanpa** ada data yang terhapus (bukti pemeriksaan `$_SERVER['REQUEST_METHOD']` dari [bab 3 §3.3](#) bekerja).

7.4 Ide Latihan Tambahan (Opsional)

23. **Tambah konfirmasi ekstra sebelum Update** — bandingkan dengan Delete yang sudah punya `confirm()`; apakah Update juga butuh konfirmasi serupa? Pertimbangkan kapan konfirmasi tambahan

benar-benar diperlukan (ingat: Update tidak destruktif seperti Delete, data lama masih "terlihat" sebelum diubah).

24. **Ubah jumlah baris per halaman** — ganti `$perPage = 5`; menjadi `10` di `buku/list.php`, amati bagaimana jumlah total halaman berubah mengikuti.

25. **Tambah pencarian di kolom lain** — misalnya perluas query di [bab 5 §5.6](#) supaya juga mencocokkan kolom `pengarang`, bukan cuma `judul` (petunjuk: gunakan `OR` di klausa `WHERE`).

26. **Terapkan pola Update/Delete ke fitur lain** — kalau kamu menambah entitas baru di proyek pribadimu nanti, coba terapkan pola CRUD yang sama persis: `list.php` (Read + pagination), `tambah.php` (Create), `edit.php` (Update), `hapus.php` (Delete) — pola 4 file ini akan terus berulang untuk hampir semua data yang perlu dikelola.

Kalau ada bagian yang masih membingungkan, terutama soal kenapa `WHERE` begitu penting di `UPDATE/DELETE`, coba baca ulang

[bab 2 §2.6](#)

dan [bab 3 §3.4](#) sambil

membayangkan skenario terburuknya: satu baris kode yang lupa `WHERE` bisa mengubah/menghapus **seluruh** data di tabel dalam sekejap — inilah kenapa kebiasaan ini penting ditanamkan sejak awal belajar SQL.